西南交通大學
Southwest Jiaotong University

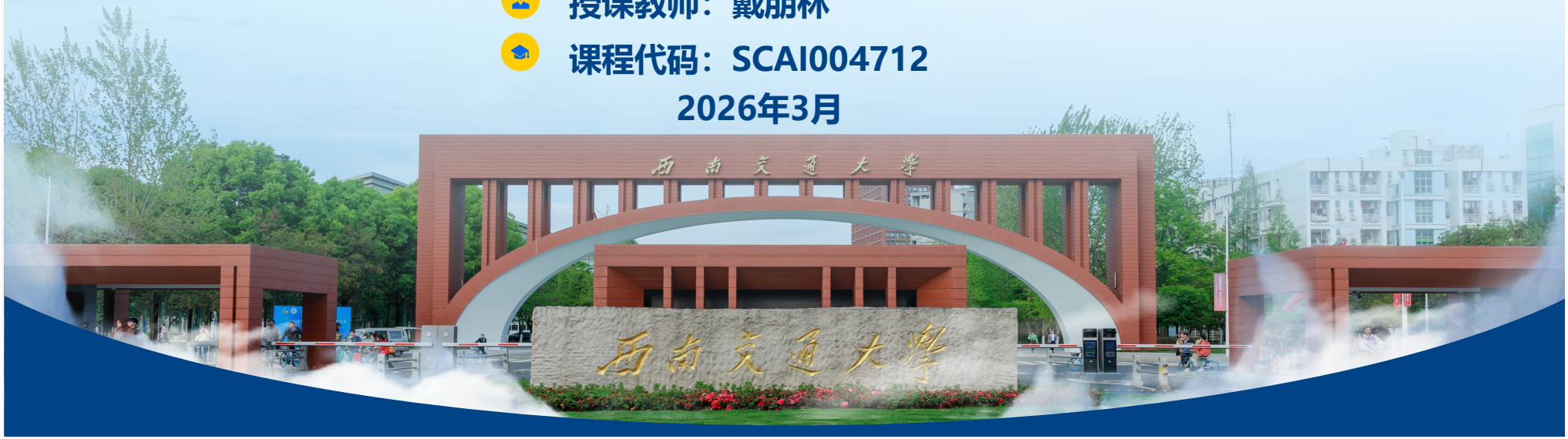
云计算技术

第5讲 Kubernetes深入与服务网格

授课教师：戴朋林

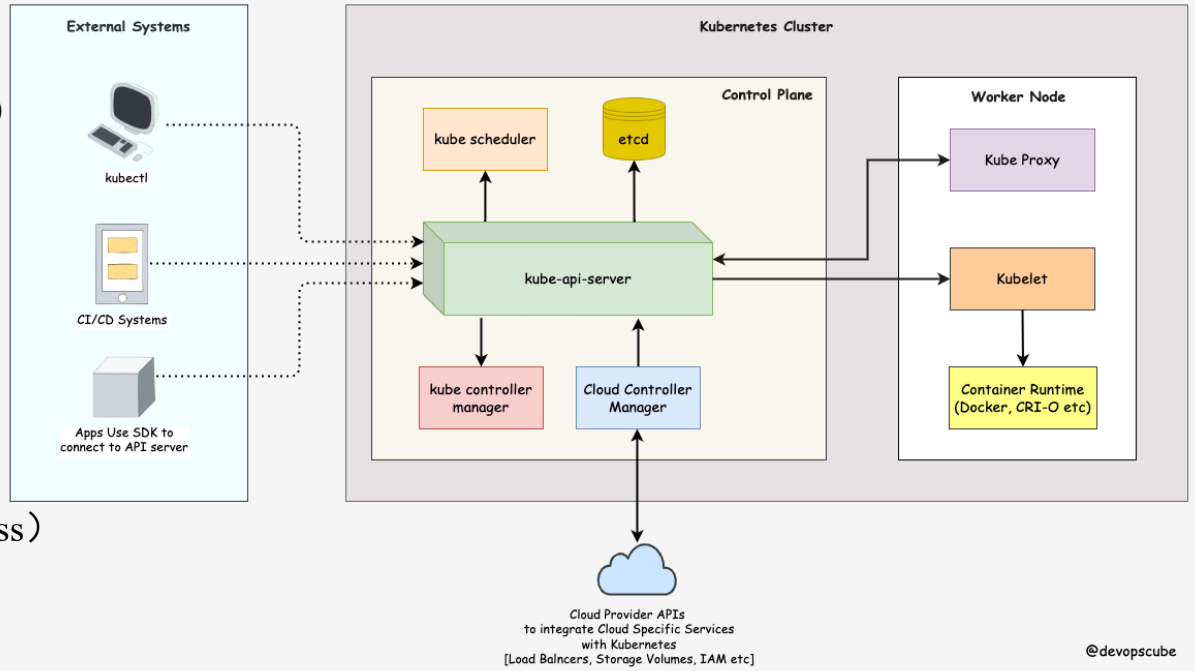
课程代码：SCAI004712

2026年3月



00 本讲内容大纲

- 资源配置与调度
  - ✓ 资源请求与限制 (Requests & Limits)
  - ✓ 调度器原理（预选与优选）
  - ✓ 节点亲和性与Pod亲和性
  - ✓ 污点与容忍（Taints & Tolerations）
  - ✓ ResourceQuota与LimitRange
- StatefulSet有状态应用管理
- ConfigMap与Secret配置管理与代码解耦
- 持久化存储体系（PV / PVC / StorageClass）
- Service Mesh与Istio服务网格架构
  - ✓ Service Mesh定义与背景
  - ✓ Istio架构（控制面/数据面）
  - ✓ Sidecar代理（Envoy）
  - ✓ 流量管理、安全mTLS、金丝雀发布
- 应用案例与演示代码



kubernetes

1 资源配置与调度

- 资源请求与限制 (Requests & Limits)
- 调度器原理（预选与优选）
- 节点亲和性与Pod亲和性
- 污点与容忍（Taints & Tolerations）
- ResourceQuota与LimitRange

01 资源请求与限制 (Requests & Limits)

西南交通大学 Southwest Jiaotong University

- 为什么需要设置资源？
  - Requests (请求)：调度器以此为依据选择节点（保底资源）。
  - Limits (限制)：容器运行时的最大上限（熔断资源）。
- 核心配置

```yaml
resources:
  requests:
    memory: "64Mi"   # 调度依据
    cpu: "250m"      # 0.25核
  limits:
    memory: "128Mi" # 超过OOM Kill
    cpu: "500m"      # 超过限流
```

01 调度器原理：预选与优选

西南交通大学 Southwest Jiaotong University

- kube-scheduler 调度流程
  - 调度（Scheduling）是指将未绑定节点的Pod分配到合适Worker节点的过程。kube-scheduler通过两阶段筛选完成调度决策。
- 两阶段调度

阶段	名称	说明
阶段1	预选（Filtering / Predicates）	排除不满足条件的节点，得到候选节点列表
阶段2	优选（Scoring / Priorities）	对候选节点评分（0-100），选择得分最高的节点

- 预选常见规则

规则	说明
PodFitsResources	节点剩余资源≥Pod的Request
PodFitsHostPorts	节点端口无冲突
MatchNodeSelector	节点标签匹配Pod的nodeSelector
NoTaintUnlessTolerated	节点Taint需被Pod Toleration匹配
CheckNodeMemoryPressure	节点无内存压力

- 优选常见规则

规则	说明
LeastRequestedPriority	优先选择剩余资源最多的节点（分散调度）
BalancedResourceAllocation	CPU和内存使用率更均衡的节点得分更高
NodeAffinityPriority	匹配亲和性规则的节点得分更高
InterPodAffinityPriority	满足Pod亲和性的节点得分更高

01 节点亲和性（Node Affinity）

西南交通大学 Southwest Jiaotong University

- 节点亲和性（Node Affinity）是一种调度约束，允许Pod声明对特定节点的偏好，基于节点标签进行匹配。它是nodeSelector的增强版，支持更复杂的表达式。
- 两种模式

模式	说明	效果
requiredDuringSchedulingIgnoredDuringExecution	硬约束——必须满足	不满足则Pod无法调度
preferredDuringSchedulingIgnoredDuringExecution	软约束——优先满足	不满足仍可调度到其他节点

- 配置示例

```yaml
affinity:
  nodeAffinity:
    requiredDuringSchedulingIgnoredDuringExecution:
      nodeSelectorTerms:
      - matchExpressions:
        - key: disk-type
          operator: In
          values: ["ssd", "nvme"] # 必须是 SSD
```

01 节点亲和性（Node Affinity）

西南交通大学 Southwest Jiaotong University

- Pod亲和性（Pod Affinity）
  - Pod亲和性允许Pod声明希望与具有特定标签的其他Pod调度到同一节点/区域，提升通信效率。
  - 示例

```
affinity:
  podAffinity:
    requiredDuringSchedulingIgnoredDuringExecution:
    - labelSelector:
        matchLabels:
          app: redis-cache
      topologyKey: kubernetes.io/hostname
```

  - 场景：Web 与 Redis 部署在一起

01 节点亲和性（Node Affinity）

西南交通大学 Southwest Jiaotong University

- Pod反亲和性（Pod Anti-Affinity）
  - Pod反亲和性允许Pod声明不希望与特定Pod在同一节点/区域，容灾（单点故障不影响整体）。
  - 示例

```
affinity:
  podAntiAffinity:
    preferredDuringSchedulingIgnoredDuringExecution:
    - weight: 100
      podAffinityTerm:
        labelSelector:
          matchLabels:
            app: my-web-app
        topologyKey: kubernetes.io/hostname
```

  - 场景：高可用分散部署

01 污点与容忍（Taints & Tolerations）

- 污点（Taint）是节点的属性，表示该节点拒绝接受特定Pod的调度。容忍（Toleration）是Pod的属性，表示该Pod可以容忍某个节点的污点。
  - 亲和性＝Pod主动选择节点（"我想去哪"）
  - 污点/容忍＝节点排斥Pod（"谁不能来"），Pod可声明豁免
- Taint效果类型

Effect	说明
NoSchedule	不调度新Pod到该节点（已运行的Pod不受影响）
PreferNoSchedule	尽量不调度，但不保证
NoExecute	不调度新Pod，且驱逐已运行的不匹配Pod

- 操作命令

```
# 添加Taint
kubectl taint nodes node1 gpu=true:NoSchedule
# 查看节点Taint
kubectl describe node node1 | grep Taint
# 移除Taint
kubectl taint nodes node1 gpu=true:NoSchedule-
```

- 典型场景
  - GPU节点：Taint为`gpu=true:NoSchedule`，只有需要GPU的Pod才容忍该Taint
  - Master节点：默认Taint `node-role.kubernetes.io/master:NoSchedule`，避免业务Pod调度到Master

01 ResourceQuota与LimitRange

西南交通大学 Southwest Jiaotong University

- ResourceQuota——命名空间级资源配额
  - ResourceQuota限制一个命名空间中可使用的总资源量。

```
# ResourceQuota 示例
spec:
  hard:
    requests.cpu: "4"
    requests.memory: "8Gi"
    pods: "10"
```

- LimitRange——Pod/容器级默认资源约束
  - LimitRange为命名空间中的Pod/容器设置默认值和范围约束。

对比	ResourceQuota	LimitRange
作用范围	命名空间总量	单个Pod/容器
功能	限制总资源、对象数量	设置默认值、范围

2 StatefulSet

有状态应用管理

02 StatefulSet有状态应用管理

西南交通大学 Southwest Jiaotong University

- StatefulSet是Kubernetes中管理有状态应用的工作负载控制器。与Deployment不同，StatefulSet为每个Pod提供稳定的网络标识和持久化存储，确保Pod的身份在重建后保持不变。
- StatefulSet vs Deployment

特性	Deployment	StatefulSet
Pod名称	随机后缀（web-7d9f8c-xk2l3）	有序固定（web-0, web-1, web-2）
网络标识	无稳定标识	稳定的DNS名称（web-0.web-svc）
存储	Pod重建后存储丢失	每个Pod绑定独立的PVC，重建后保留
启动/停止顺序	并行	有序（0→1→2启动，2→1→0停止）
适用场景	无状态应用（Web API）	有状态应用（数据库、消息队列）

02 核心组件：Headless Service

西南交通大学 Southwest Jiaotong University

- 为什么需要 Headless Service?
  - 普通 Service 有 ClusterIP，做负载均衡。
  - Headless Service(`clusterIP: None`)不分配 IP，直接返回后端 Pod 的 IP 列表。
  - 它配合 StatefulSet 生成稳定的 DNS 记录：`pod-name.service-name`。

```yaml
apiVersion: v1
kind: Service
metadata:
  name: mysql
spec:
  clusterIP: None  # 关键点
  selector:
    app: mysql
```

02 核心特性：VolumeClaimTemplates

西南交通大学 Southwest Jiaotong University

- 自动化存储绑定
  - Deployment 所有 Pod 共享一个 PVC（通常用于只读）。
  - StatefulSet 需要每个 Pod 有自己独立的 PVC。

```
volumeClaimTemplates:  # 为每个 Pod 生成专属 PVC
- metadata:
    name: data
  spec:
    accessModes: [ "ReadWriteOnce" ]
    storageClassName: "fast-storage"
    resources:
      requests:
        storage: 1Gi
```

3 ConfigMap与Secret

配置与代码解耦

03 ConfigMap：非敏感配置

- ConfigMap是Kubernetes中用于存储非敏感配置数据的API对象，以键值对形式存储，实现配置与应用代码解耦。
- 使用场景
  - 环境变量：`DB_HOST`, `LOG_LEVEL`
  - 配置文件：`nginx.conf`, `redis.conf` (挂载为 Volume)

```
# 挂载配置文件
volumes:
- name: config-volume
  configMap:
    name: nginx-config
```

- 使用方式

方式	说明
环境变量注入	envFrom 或单个 valueFrom
卷挂载为文件	将ConfigMap挂载为目录中的文件
命令行参数	通过环境变量传递给命令

03 ConfigMap：非敏感配置

西南交通大学 Southwest Jiaotong University

- ConfigMap使用示例

方式1：作为环境变量

```yaml
spec:
  containers:
  - name: app
    image: myapp:v1
    envFrom:                          # 注入所有键值对
    - configMapRef:
        name: app-config
    env:                              # 注入单个键值对
    - name: DATABASE_HOST
      valueFrom:
        configMapKeyRef:
          name: app-config
          key: DB_HOST
```

方式2：挂载为文件

```yaml
spec:
  containers:
  - name: nginx
    image: nginx:alpine
    volumeMounts:
    - name: config-vol
      mountPath: /etc/nginx/conf.d
  volumes:
  - name: config-vol
    configMap:
      name: nginx-conf
```

- 热更新
  - 环境变量方式：修改ConfigMap后，需要重启Pod才能生效
  - 卷挂载方式：kubelet周期性同步（约1分钟），自动更新文件内容，无需重启
  - 应用需要自行实现配置热加载逻辑（如watch文件变化）

03 Secret：敏感信息管理

西南交通大学 Southwest Jiaotong University

- Secret是Kubernetes中用于存储敏感信息（密码、Token、证书等）的API对象。数据以Base64编码存储（注意：非加密，需配合etcd加密使用）。

类型	用途
Opaque	通用密钥（默认类型）
kubernetes.io/tls	TLS证书（tls.crt + tls.key）
kubernetes.io/dockerconfigjson	Docker Registry认证信息
kubernetes.io/service-account-token	ServiceAccount Token

- 创建方式

```
# 命令行创建
kubectl create secret generic db-secret \
  --from-literal=username=admin \
  --from-literal=password=S3cureP@ss
```

```
# YAML创建（值需Base64编码）
apiVersion: v1
kind: Secret
metadata:
  name: db-secret
type: Opaque
data:
  username: YWRtaW4=          # echo -n "admin" | base64
  password: UzNjdXJlUEBzcw== # echo -n "S3cureP@ss" | base64
```

- Secret vs ConfigMap
  - 存储内容：密码、Token、TLS 证书 vs 普通配置。
  - 编码方式：Base64 编码（注意：Base64 不是加密，只是编码）。
  - 使用方式：挂载为 tmpfs (内存卷)，不会写入节点磁盘。

4 持久化存储体系

PV / PVC / StorageClass

04 K8s 存储架构概览

西南交通大学 Southwest Jiaotong University

- 问题：容器存储的临时性
  - 容器文件系统是临时的——容器重启或删除后数据丢失
  - 有状态应用（数据库、文件服务）需要持久化存储
- Kubernetes存储三层架构
  - Pod → PVC（存储请求） → PV（存储资源） → 底层存储（云盘/NFS/Ceph）

概念	全称	说明
PV	PersistentVolume	集群级别的存储资源，由管理员预先创建或动态供给
PVC	PersistentVolumeClaim	Pod对存储的请求声明（大小、访问模式）
StorageClass	-	定义存储的类型和供给策略，支持动态创建PV

  - 访问模式

模式	缩写	说明
ReadWriteOnce	RWO	单节点读写（最常用）
ReadOnlyMany	ROX	多节点只读
ReadWriteMany	RWX	多节点读写（需NFS/CephFS等共享存储）

  - 回收策略

策略	说明
Retain	PVC删除后PV保留（手动回收）
Delete	PVC删除后PV和底层存储一并删除
Recycle	已弃用（清空数据后重新使用）

04 静态供给 (Static Provisioning)

西南交通大学 Southwest Jiaotong University

- 传统流程
  - 管理员手动创建 10 个 PV（预先分配）。
  - 开发人员创建 PVC。
  - K8s 寻找匹配的 PV 进行绑定。

```yaml
# 1. 管理员创建PV
apiVersion: v1
kind: PersistentVolume
metadata:
  name: nfs-pv
spec:
  capacity:
    storage: 50Gi
  accessModes:
    - ReadWriteMany
  persistentVolumeReclaimPolicy: Retain
  nfs:
    server: 192.168.1.100
    path: /data/k8s-volumes
---
```

```yaml
# 2. 开发者创建PVC
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: app-data-pvc
spec:
  accessModes:
    - ReadWriteMany
  resources:
    requests:
      storage: 10Gi
```

```yaml
# 3. Pod使用PVC
apiVersion: v1
kind: Pod
metadata:
  name: app
spec:
  containers:
  - name: app
    image: myapp:v1
    volumeMounts:
    - name: data
      mountPath: /app/data
  volumes:
  - name: data
    persistentVolumeClaim:
      claimName: app-data-pvc
```

04 动态供给 (Dynamic Provisioning)

西南交通大学 Southwest Jiaotong University

- 静态供给的问题
  - 管理员需要提前创建大量PV
  - PV大小和PVC请求不一定匹配（浪费资源）
  - 无法自动化
- 动态供给（Dynamic Provisioning），StorageClass定义存储的类型和供给者（Provisioner），当PVC请求匹配某个StorageClass时，自动创建对应的PV。

```yaml
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: fast-ssd
provisioner: kubernetes.io/aws-ebs    # 供给者
parameters:
  type: gp3                           # AWS EBS gp3磁盘类型
  fsType: ext4
reclaimPolicy: Delete
volumeBindingMode: WaitForFirstConsumer  # 延迟绑定
```

04 动态供给 (Dynamic Provisioning)

西南交通大学 Southwest Jiaotong University

- StorageClass与动态供给
  - PVC使用StorageClass

```yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: db-data
spec:
  accessModes: ["ReadWriteOnce"]
  storageClassName: fast-ssd    # 指定StorageClass
  resources:
    requests:
      storage: 100Gi
```

  - 动态供给流程

PVC请求（storageClassName: fast-ssd, 100Gi）
→ StorageClass（provisioner: aws-ebs）
→ 自动创建100Gi EBS卷
→ 创建PV并绑定PVC
→ Pod使用PVC

5 Service Mesh与Istio服务网格架构

- Service Mesh的定义与背景
- Istio架构
- Sidecar代理（Envoy）
- 流量管理
- 安全mTLS
- 金丝雀发布

05 定义与背景

西南交通大学 Southwest Jiaotong University

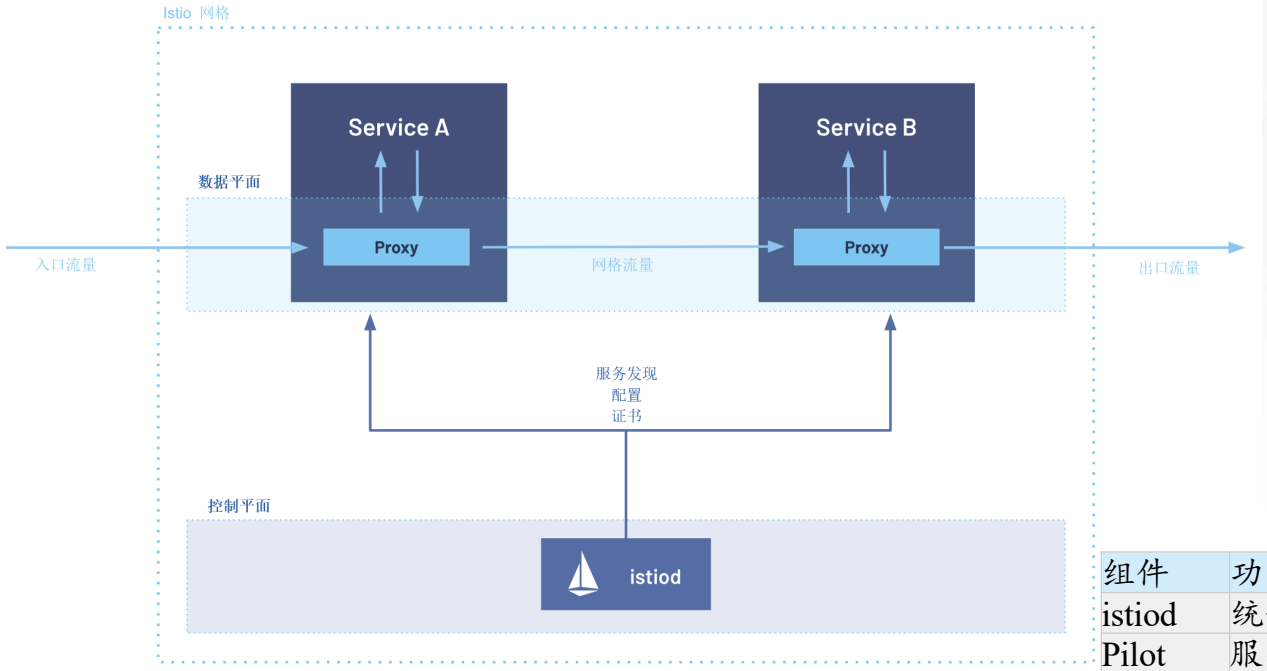
- 问题：微服务通信的复杂性，随着微服务数量增长，服务间通信面临诸多挑战：
  - 流量管理（负载均衡、重试、超时）
  - 安全（加密通信、身份认证）
  - 可观测性（链路追踪、指标、日志）

  传统方式：在每个服务中嵌入SDK（如Spring Cloud、Hystrix）——语言绑定，侵入性强
- Service Mesh定义
  - 服务网格（Service Mesh）是一个专门用于处理服务间通信的基础设施层。它通过在每个服务旁部署一个轻量级网络代理（Sidecar），以透明的方式处理服务发现、负载均衡、故障恢复、指标采集和安全通信。——William Morgan, Buoyant CEO, 2017
  - 核心价值

传统SDK方式	Service Mesh
业务代码中嵌入通信逻辑	通信逻辑下沉到基础设施
语言绑定（Java SDK无法用于Go服务）	语言无关（Sidecar代理）
升级SDK需要修改并重新部署所有服务	升级代理不影响业务代码
每个服务单独实现熔断/限流	统一的策略管理

05 Istio架构

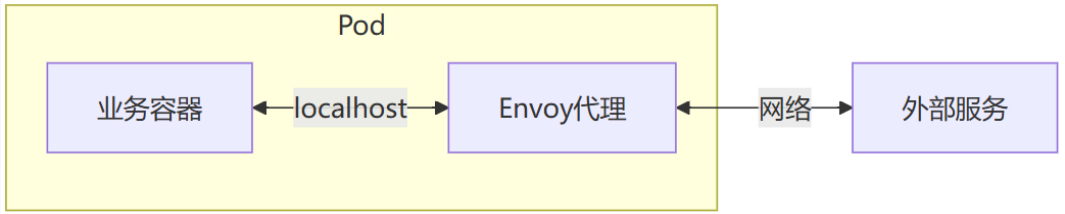
- Istio是由Google、IBM和Lyft共同开发的开源服务网格平台，是目前最流行的Service Mesh



组件	功能
istiod	统一的控制面（合并了Pilot/Citadel/Galley）
Pilot	服务发现、流量管理规则下发
Citadel	证书管理、mTLS加密
Galley	配置验证和分发
Envoy	数据面代理（Sidecar），所有流量经过Envoy

05 Sidecar代理（Envoy）

- 什么是Sidecar模式？
  - 在Kubernetes中，Istio通过自动注入（Injection）的方式，为每个Pod添加一个Envoy代理容器（Sidecar）。所有进出Pod的网络流量都会被Envoy拦截和处理。



- Envoy简介
  - Envoy是由Lyft开发的高性能L4/L7代理，是CNCF毕业项目。它是Istio数据面的核心。

能力	说明
L7路由	基于HTTP头、路径、权重的智能路由
负载均衡	Round Robin、Least Connection、一致性哈希等
熔断	连接数限制、请求数限制
重试	自动重试失败请求
超时	请求级别超时控制
TLS	自动mTLS加密通信
可观测性	自动生成请求指标（Prometheus）、分布式追踪（Jaeger）

05 流量管理

西南交通大学 Southwest Jiaotong University

- 路由规则定义
  - 决定请求到达 Service 后，去哪个版本的 Pod。

```yaml
spec:
  hosts:
  - reviews
  http:
  - match:
    - headers:
        end-user:
          exact: jason
    route:
    - destination:
        host: reviews
        subset: v2  # 测试用户去 v2
  - route:
    - destination:
        host: reviews
        subset: v1  # 其他人去 v1
```

- 定义子集与策略
  - Subsets：将一个 Service 的 Pod 划分为 v1, v2。
  - TrafficPolicy：负载均衡策略、连接池、熔断。

```yaml
spec:
  host: reviews
  subsets:
  - name: v1
    labels:
      version: v1
  trafficPolicy:
    loadBalancer:
      simple: RANDOM  # 随机负载均衡
```

05 Istio安全：mTLS

西南交通大学 Southwest Jiaotong University

- mTLS（Mutual TLS）是双向TLS认证。传统TLS只有客户端验证服务端证书，mTLS要求双方互相验证对方身份。
- Istio mTLS工作流程
  - Istio Citadel为每个服务自动签发X.509证书
  - Envoy自动使用证书建立mTLS连接
  - 应用代码无需任何修改——加密对应用透明

Service A (Envoy) ←— mTLS加密通道 —→ Service B (Envoy)

↑ Citadel签发的客户端证书

↑ Citadel签发的服务端证书

05 金丝雀发布（Canary Release）与流量灰度

- 金丝雀发布是一种渐进式发布策略，先将少量流量导向新版本，观察无异常后逐步扩大新版本的流量比例，直到全量切换。
- Istio实现金丝雀发布
  - 阶段1: v1=100%, v2=0% → 部署v2，不导入流量
  - 阶段2: v1=90%，v2=10% → 10%流量到v2，观察指标
  - 阶段3: v1=50%，v2=50% → 扩大到50%
  - 阶段4: v1=0%，v2=100% → 全量切换到v2
- 对比其他发布策略

策略	原理	风险	回滚速度
滚动更新	逐步替换旧Pod	中等	慢（需回滚Deployment）
蓝绿部署	两套完整环境切换	低	快（切换路由）
金丝雀发布	按比例分流	最低	最快（调整权重）
A/B测试	按条件（用户/地区）分流	低	快

05 金丝雀发布（Canary Release）与流量灰度

西南交通大学 Southwest Jiaotong University

- Istio的优势
  - 流量权重精确控制：可以精确到1%的流量比例
  - 基于请求内容路由：根据Header、Cookie、URL等条件分流
  - 实时切换：修改VirtualService即刻生效
  - 回滚零成本：调整权重回到100% v1即可

6 应用案例 Python Client 与配置生成

06 演示 1：Python 操作 Kubernetes

西南交通大学 Southwest Jiaotong University

- 环境准备与连接

```python
from kubernetes import client, config

# 加载 kubeconfig (默认 ~/.kube/config)
config.load_kube_config()

# 创建 API 客户端实例
v1 = client.CoreV1Api()
apps_v1 = client.AppsV1Api()

print("K8s API 连接成功")
```

06 演示 2：创建 ConfigMap 与 Pod

西南交通大学 Southwest Jiaotong University

- 动态创建资源

```
# 1. 定义 ConfigMap 对象
cm = client.V1ConfigMap(
    metadata=client.V1ObjectMeta(name="py-config"),
    data={"app_mode": "prod", "db_host": "10.1.1.1"}
)

# 2. 提交到 API Server
v1.create_namespaced_config_map(namespace="default", body=cm)
print("ConfigMap Created.")

# 3. 创建 Pod 使用该 ConfigMap (代码略，展示思路)
# ... envFrom: configMapRef ...
```

06 演示 3：生成 Istio 灰度配置

- 自动化运维脚本

```python
import yaml

def create_canary_yaml(service, v1_ratio, v2_ratio):
    vs = {
        'apiVersion': 'networking.istio.io/v1beta1',
        'kind': 'VirtualService',
        'metadata': {'name': f'{service}-route'},
        'spec': {
            'hosts': [service],
            'http': [{
                'route': [
                    {'destination': {'subset': 'v1'}, 'weight':
v1_ratio},
                    {'destination': {'subset': 'v2'}, 'weight':
v2_ratio}
                ]
            }]
        }
    }
    return yaml.dump(vs)

# 生成 80/20 分流配置
print(create_canary_yaml("reviews", 80, 20))
```

07 课堂思考与作业

西南交通大学 Southwest Jiaotong University

- 思考题
  - 如果一个 Pod 需要 10G 内存，但集群里最大的节点只有 8G，调度器会怎么做？(Pending)
  - StatefulSet 删除后，对应的 PVC 会自动删除吗？(默认不会，Retain)
  - 在没有 Service Mesh 之前，你们是如何做灰度发布的？(Nginx 权重？)
- 作业
  - 编写一个 StatefulSet YAML，部署 3 副本的 Redis。
  - 使用 Python 脚本读取该 StatefulSet 的状态。

08 本讲知识回顾

西南交通大学 Southwest Jiaotong University

核心模块	关键知识点
调度	Request/Limit, Affinity, Taint/Toleration
有状态	StatefulSet, Headless Service, PVC Templates
存储	PV (资源), PVC (请求), StorageClass (自动创建)
Istio	Sidecar, VirtualService (路由), mTLS (安全)

09 下次课预告 &Q&A

西南交通大学 Southwest Jiaotong University

- 下次课：第6讲 消息队列与事件驱动架构
  - 消息队列基础概念（点对点 / 发布订阅）
  - Apache Kafka架构与实践
  - RabbitMQ核心概念
  - 事件驱动架构（EDA）
  - Redis Pub/Sub与Stream
- 预习建议：
  - 了解消息队列的基本概念和应用场景
  - 安装Docker，准备运行Kafka/RabbitMQ容器